PAYRECON

Build Plan

How PayRecon was planned, built and checked, step by step, in plain English.

Payment ledger + bank reconciliation · Java 17 · MySQL · Hadoop MapReduce · Hive · MongoDB

At a glance

Goal	An SDE-interview project on MapReduce, Hive, MongoDB and MySQL. No Pig, no machine learning, and every tool has a real job
Status	Complete: built, tested and documented
Checks	27 unit tests, the answer-key check, the tie-out, a 9-rule ledger audit and a stress test: all pass
Size	About 6,300 lines of Java, 7 Hive scripts and 15 MySQL tables

This document explains, in plain English, how PayRecon was planned and built:

- the goal
- the idea we picked and why
- how the computer was set up
- the order in which the parts were built
- how each part was checked
- the problems we found and fixed on the way

To learn how the finished system works inside, read *Project Explained* (PayRecon-Project-Explained.pdf).

Contents

1. The goal 3
2. Choosing the idea 3
3. The plan at a glance 3
4. Setting up the computer 5
5. The build, step by step 6
6. How the data is stored (MySQL) 7
7. Important decisions, and why 8
8. How we check that it is right 8
9. Problems found during the build, and how they were fixed 9
9.1 Money stuck in the suspense account . . . 9
9.2 The wrong copy of a duplicate . . . 9
9.3 Indian digit grouping . . . 9
10. Final results 9
11. What a real company would add next 10

1. The goal

Build one project that:

- **shows real backend skills** for software developer (SDE) interviews: database transactions, many users at the same time, safe retries, and processing a lot of data
- **uses the tools from the NoSQL course project** (Hadoop MapReduce, Hive, MongoDB and MySQL) but **not Pig**
- **gives every tool a real job**, with no tool added just to tick a box
- **uses no machine learning at all**
- **copies a real company's product**, so it is easy to explain in an interview

2. Choosing the idea

Several product ideas were compared before building anything:

Idea	Copies the product of	Decision
SnapSentinel	Rubrik (backup security)	strong demo, but more data processing than backend work
DedupVault	Rubrik, Cohesity (storage)	great algorithms, but Hive would only have had a weak job
LakeForge	Databricks (Delta Lake)	very deep, but the hardest to finish and very specialised
BlastRadius	Snyk, GitHub Dependabot	real public data, but that data needs a lot of cleaning
PayRecon	**Stripe, Razorpay (ledger + reconciliation)**	**chosen**: transactions, safe retries and locking are exactly what SDE interviews ask about, and everyone understands money

Ideas built around fraud scoring, anomaly detection or drift detection were dropped early, because their core is machine learning.

3. The plan at a glance

PayRecon has two halves:

- **A ledger.** This is the official record of every payment, refund and payout. It is kept with double-entry bookkeeping in MySQL, behind a small web API.
- **A reconciliation engine.** Every month it checks the files the banks send against the ledger and explains every difference.

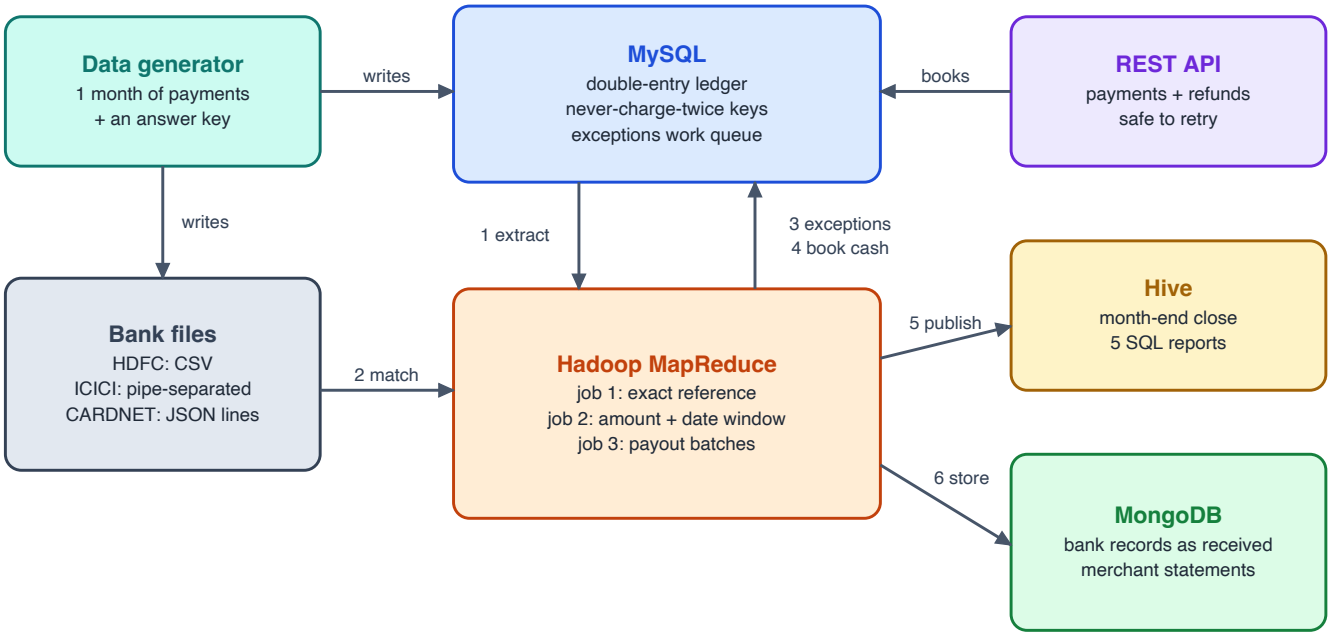


Figure 1. The big picture. The numbers 1 to 6 are the six steps of every reconciliation run.

Each tool has one clear job:

Tool	Its job
MySQL	the ledger, the "never charge twice" keys, and the analysts' work queue: everything that needs safe transactions
Hadoop MapReduce	matching the ledger against the bank files, in three jobs
Hive	month-end reports in SQL over all the result files
MongoDB	bank records exactly as they arrived (every bank uses a different format), and ready-made merchant statements

4. Setting up the computer

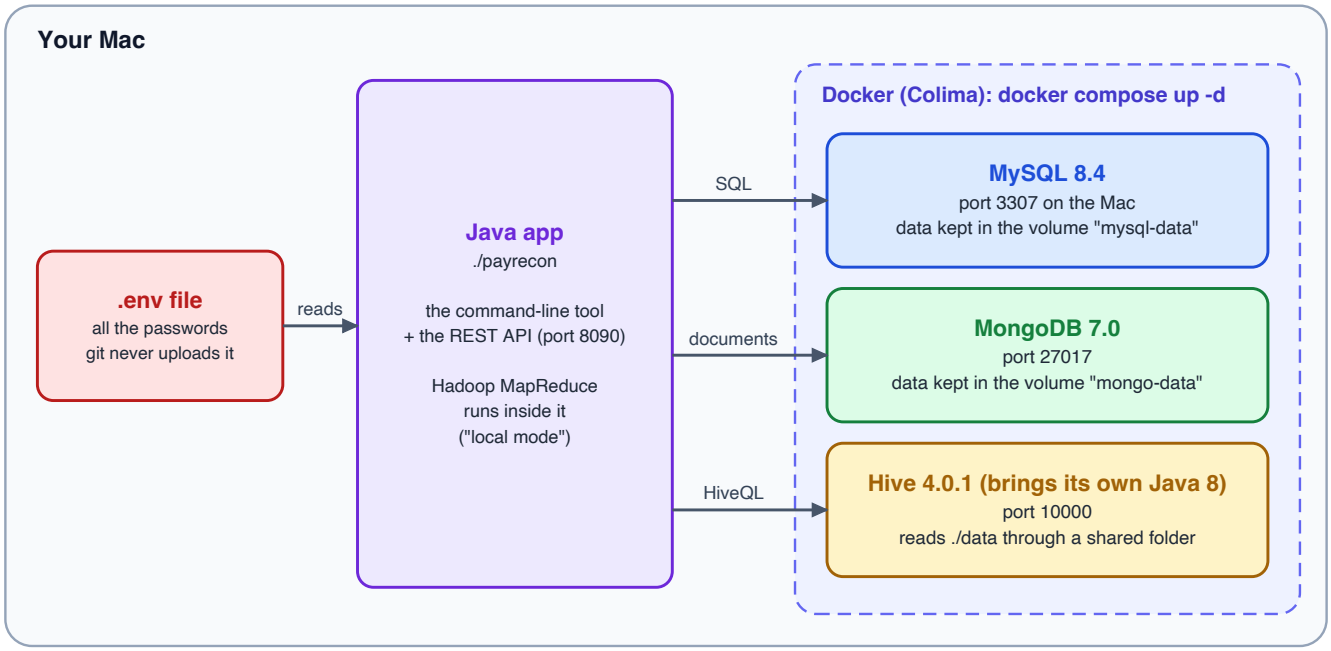


Figure 2. The setup: one Java program on the Mac, plus three databases in Docker containers.

- **Docker Compose** starts MySQL, MongoDB and Hive as three **containers**: small isolated boxes, each with exactly the right version and everything it needs. Nothing has to be installed on the Mac itself, and anyone can start the same setup with one command: `docker compose up -d`.
- **Hive needed this most.** Hive only runs on Java 8, while the Mac has Java 17. The Hive container brings its own Java.
- **Ports:**
  - MySQL uses port **3307**, so it never clashes with a MySQL already installed on the computer (3306).
  - The API uses **8090**, because 8080 was already taken.
- **Passwords** are kept in a `.env` file that git never uploads. The NASA course project had a password written in its code; this project fixes that.
- **MapReduce runs inside the Java program** in Hadoop's "local mode", so no Hadoop cluster is needed. The same code would also run on a real cluster.
- **One trap:** the project folder's name contains a space ("Nosql project"), and some Hadoop tools dislike that. Hadoop's scratch folder therefore lives in `/tmp`.

5. The build, step by step

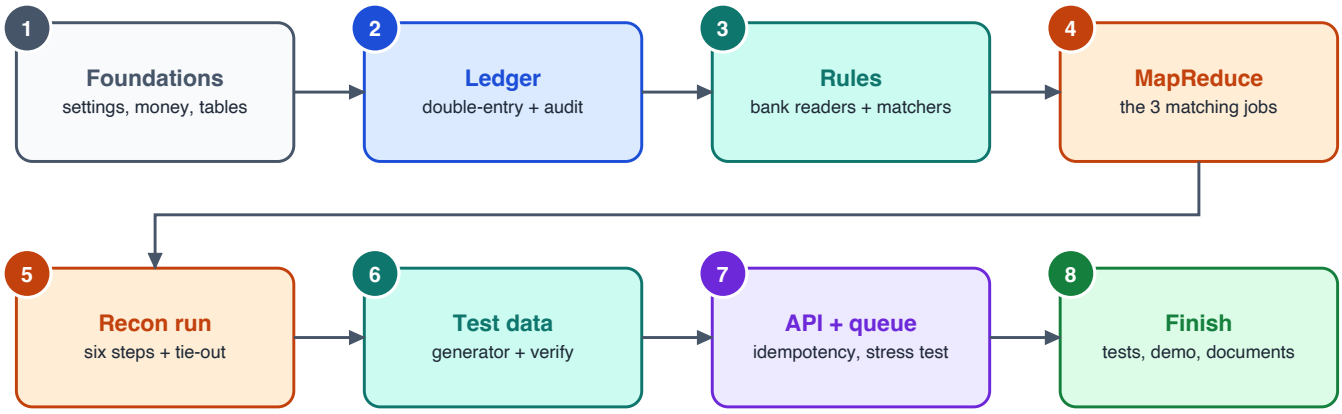


Figure 3. The eight build steps. Each step was checked before the next one started.

The project was built in eight steps. Each step was checked before the next one started.

Step	What was built	How it was checked
1. Foundations	settings from `.env`, money helpers (amounts in paise), a database helper that retries after deadlocks, the 15 MySQL tables	compiles; tables created
2. The ledger	journal entries where debits always equal credits; accounts, payments, refunds, payouts	the ledger audit (9 rules)
3. Reading bank files + matching rules	**one** reader for all three bank formats; the three matching rules written as plain functions	unit tests
4. The MapReduce jobs	job 1: exact reference; job 2: amount + date window; job 3: payout batches	first full run
5. The reconciliation run	the six steps: extract, match, load, book the cash, Hive, MongoDB; plus the tie-out check	every bank ties out
6. Test data with planted problems	a generator that creates one month of business plus an answer key; the `verify` command	verify PASS
7. Queue, API, stress test	the analysts' work queue, a REST API with idempotency keys, a concurrency stress test	stress test PASS
8. Tests, demo, documents	27 unit tests, `run-demo.sh`, RUN.md, these documents and PDFs	the one-command demo runs end to end

6. How the data is stored (MySQL)

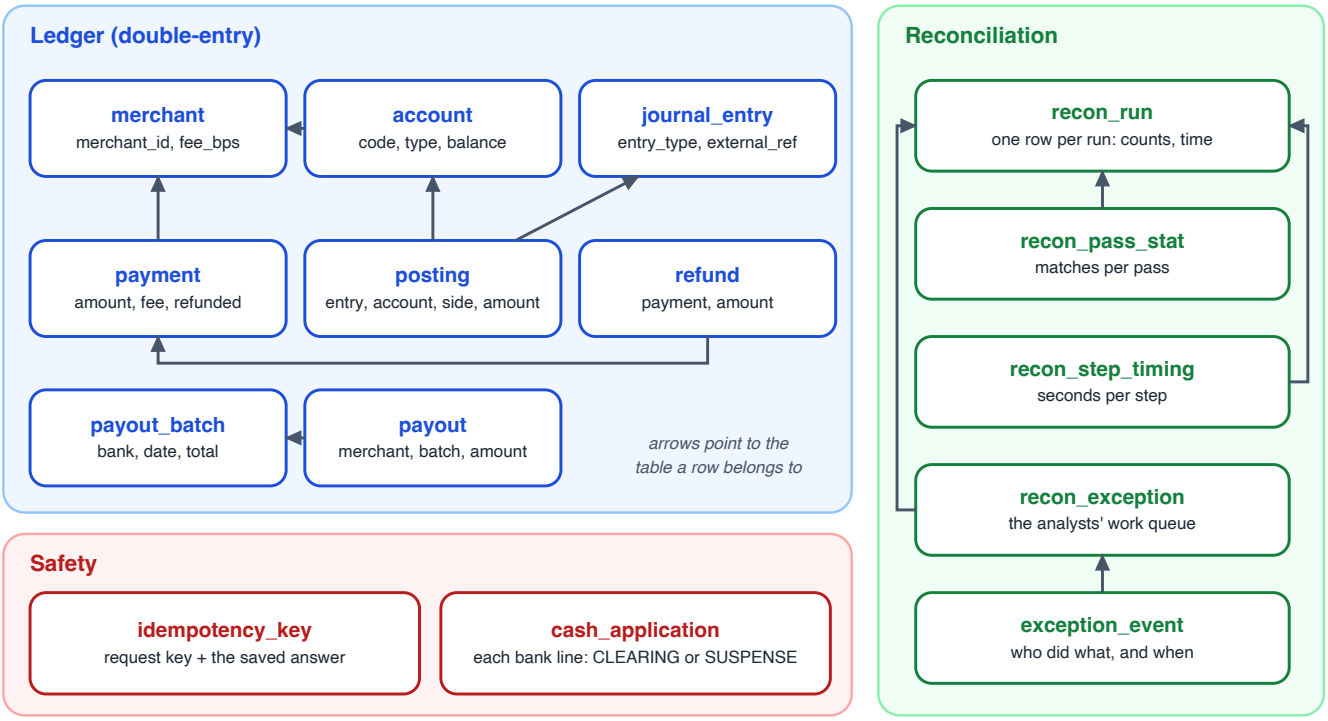


Figure 4. The 15 MySQL tables in three groups. Arrows point from a row to the row it belongs to (foreign keys).

The 15 MySQL tables fall into three groups:

- **The ledger:**
  - `merchant` and `account`
  - `journal_entry` and `posting`, the double-entry core. A **journal entry** is one event, such as "payment captured". Each **posting** is one line of it: an account, debit or credit, and an amount.
  - the business records `payment`, `refund`, `payout` and `payout_batch`, each linked to its journal entry
- **Safety:**
  - `idempotency_key` remembers every API request key, so a retried request is answered from memory instead of charging twice.
  - `cash_application` remembers, for every bank line, where its money was booked.
- **Reconciliation:**
  - `recon_run`, one row per run with counts and timing, plus `recon_pass_stat` and `recon_step_timing`
  - `recon_exception`, the analysts' work queue
  - `exception_event`, the audit trail of who did what

7. Important decisions, and why

Decision	Why	What we avoided
Store money as whole paise (a `long`)	computers cannot store most decimal fractions exactly	`float` / `double`
Double-entry ledger	every rupee has a source and a destination, so one SQL check finds bugs anywhere	a single balance column
Save the idempotency key in the same transaction as the payment	a retry can never charge twice, even after a crash	saving the key separately
Lock accounts in a fixed order, and update balances before writing postings	two payments can never block each other forever (deadlock)	locking in random order
Matching rules as plain functions, separate from the Hadoop code	tested in milliseconds; could move to Spark unchanged	logic buried inside mappers
Never guess when several payments fit one bank line	a wrong match hides real money; a person decides instead	picking the "closest"
One reader for all bank formats, used by MapReduce and MongoDB	the reading rules can never drift apart (the NASA project had 14 copies of one regex)	copying parsing code
Track cash booking for every bank line	re-running a reconciliation is always safe	one entry per bank per day (this caused a bug, see 9.1)
Hive tables are "external" (they read our files)	Hive only reads; the Java app owns the files	loading data into Hive
Start the runtime clock before reading any input	honest timing	starting it later (a NASA project bug)

8. How we check that it is right

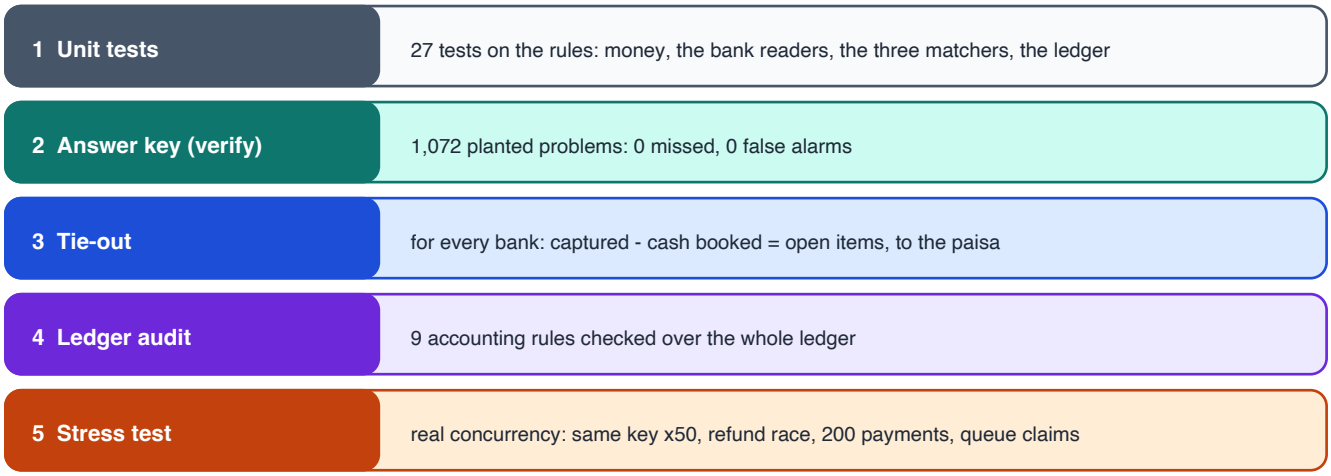


Figure 5. Five independent checks. Each one catches a different kind of mistake.

There are five independent checks, and each one catches different kinds of mistakes:

1. **Unit tests** (27): money maths, all three bank readers, the three matching rules, the ledger rules.
2. **The answer key** (`verify`): every problem planted in the bank files must be found, and nothing extra reported.
3. **The tie-out**: for each bank, what we captured minus the cash the bank paid must equal the open items, to the paisa.
4. **The ledger audit**: 9 accounting rules over the whole ledger.
5. **The stress test**: correctness when many requests arrive at the same moment.

9. Problems found during the build, and how they were fixed

9.1 Money stuck in the suspense account

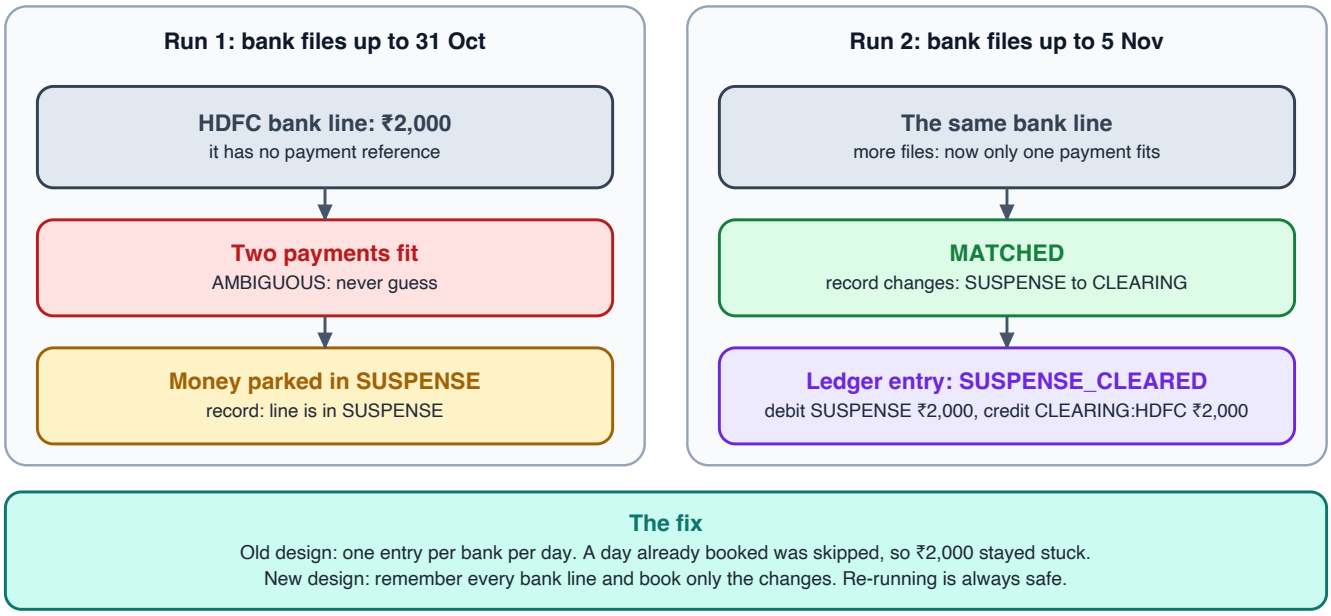


Figure 6. Why cash is tracked per bank line: money parked in suspense moves to clearing once it is identified.

- **What happened:**
  - In the first run, two ₹1,000 bank lines had no reference, and two payments fitted each of them. The engine refused to guess, so the money was parked in **suspense**, the account for money we cannot identify yet.
  - In the second run, with more bank files, each line had only one possible payment and was matched.
  - The money still stayed in suspense, and the tie-out showed a ₹2,000 difference.
- **Why:** the first design booked one entry per bank per day, and skipped days it had already booked.
- **Fix:** the `cash_application` table now remembers **every bank line**, and each run books only the *changes*. When a line moves from suspense to matched, the ledger gets a "suspense cleared" entry, which is exactly what accountants do. Two new audit checks prove the ledger and this record always agree.

9.2 The wrong copy of a duplicate

When a bank sent the same payment twice, the engine had to call the first line the original. Line ids end with a line number, and sorting them as text puts `file:10` before `file:9`, so sometimes the wrong copy was chosen. Lines are now sorted by date, then file name, then the line **number**, and a unit test guards it.

9.3 Indian digit grouping

Amounts printed as `₹63,077,555.57` instead of `₹6,30,77,555.57`. Java's number formatter cannot do Indian lakh/crore grouping, so a small function now builds it by hand. It is covered by a unit test.

10. Final results

Test month: October 2026, 1,000 shops, 2,00,000 payments (₹25.1 crore), on a laptop.

What	Result
Generate the month (ledger + 105 bank files)	13–15 seconds
One reconciliation, all six steps	about 22 seconds (MapReduce: under 8 seconds)
Payments matched automatically (of those due)	99.83%, rising to 99.92% after the next week's files

What	Result
Planted problems found	all 1,072: 0 missed, 0 false alarms
Tie-out	every bank, to the paisa, in every run
50 identical requests at the same moment	exactly 1 payment created
20 refunds racing on one ₹1,000 payment	exactly ₹1,000 refunded, never more
The one-command demo (`./run-demo.sh`)	runs end to end

11. What a real company would add next

- **Bigger scale:** run the matching on **Spark** (a newer engine that keeps data in memory) or a real Hadoop cluster.
- **Speed:** add a **connection pool**, so the API reuses database connections instead of opening a new one per request.
- **Busy accounts:** split the very busy accounts (fees, bank clearing) into several sub-accounts, so payments do not queue behind each other.
- **Money rules:**
  - support more than one currency
  - handle payments from earlier months that settle late
- **Live data:** receive bank events live (for example through **Kafka**) instead of daily files.
- **Security and monitoring:**
  - add logins and the card-security rules (**PCI-DSS**)
  - send alerts when the tie-out is not exact